TP N.º 8 - Food Store en la feria: la base de la tablet y el cierre de la capa de datos

Base de Datos II - Integrante: Mateo Algañaraz - Repositorio MateoAlganaraz/BaseDeDatos2, carpeta tp10/ (rama tp-u8) - PostgreSQL 16.15 y SQLite 3.50.4 - Todo con datos de laboratorio inventados

1.a Qué datos viajan a la tablet

Viaja (columna de v_snapshot_movil)	Origen en PostgreSQL	No viaja	Por qué
id	producto.id_producto	descripcion, id_categoria	La tablet solo muestra y vende; no los necesita.
nombre	producto.nombre	cliente (nombre, email, teléfono)	Datos personales: regla 2 de la consigna. La tablet no los necesita para vender.
precio_centavos	precio_lista × 100 (entero)	pedido y detalle_pedido	Las ventas las registra el servidor; la tablet nunca confirma ventas.
stock (stock_visto en la tablet)	producto.stock	usuarios, contraseñas, auditoría	Credenciales y datos de seguridad no salen del servidor.
disponible / eliminado	activo y deleted_at (lápida)		

La fuente de verdad es PostgreSQL. La tablet guarda una copia fechada (copiado_en) para vender sin señal. El stock que ve no es una reserva y el precio puede cambiar: por eso cada borrador pasa por el semáforo antes de convertirse en una venta, y esa conversión ocurre en el servidor, no en la tablet. Las bajas lógicas viajan como lápidas para que la tablet las deje de ofrecer sin borrar filas que los borradores referencian.

Instantánea inicial (2.a): 10.000 productos exportados con `\copy` e importados con `.import --csv --skip 1`; 9.600 disponibles. Fecha de la copia: **2026-11-02T14:22:53Z**. La base de la tablet (1.b) tiene cuatro tablas STRICT con `PRAGMA foreign_keys = ON`; las pruebas 1.c (COMMIT, ROLLBACK, C1 sin servidor y C7) están en evidencia/03_pruebas_sqlite.txt.

2.c Los casos del semáforo

Orden de cada caso: borrador en la tablet → cambio preparado en la **copia de pruebas** del servidor (foodstore_tp10_lab) → nueva instantánea → `cargar_snapshot.sql` → `semaforo.sql`. El orquestador run_tp10.sh registra cada comando.

Caso	Comando (preparación)	Resultado	Explicación
C2	casos/c2_borrador.sql; servidor/c2_subir_precio.sql (+5 % al producto 101); semáforo; casos/c2_aceptar.sql; semáforo	CONFLICTO: producto 101: precio visto 383700, actual 402885 (centavos). Luego, solicitud nueva: SIMULADO_APROBADO	No hay rojo, pero el precio visto difiere del actual. El cliente acepta el precio nuevo: la tablet actualiza el precio visto y envía una solicitud nueva. La que quedó en CONFLICTO se conserva como registro.
C3	casos/c3_borrador.sql (stock visto + 6 unidades del 106)	RECHAZADO: producto 106: pide 170, stock 164	El stock que vio la tablet no es una reserva. El semáforo compara la cantidad con el stock del servidor.
C4	casos/c4_borrador.sql; servidor/c4_baja_logica.sql (deleted_at del 109); semáforo; casos/c4_catalogo.sql	RECHAZADO: producto 109: dado de baja. En la tablet, 0 en el catálogo disponible y la fila con eliminado = 1	La baja viaja como lápida. Al recargar, el producto desaparece del catálogo, pero su fila queda porque el borrador viejo la referencia. Un borrador nuevo ya no puede incluirlo (trigger).
C5	casos/c5_borrador.sql (110 × 3 y 111 × 1)	SIMULADO_APROBADO	Existencia, disponibilidad, stock y precio coinciden. Es una simulación: no crea ningún pedido en PostgreSQL.
C6	casos/c6_reenviar.sql (mismo UUID de C5); semáforo	UNIQUE constraint failed: cola_sync.solicitud_uuid; la segunda corrida evalúa 0 solicitudes	solicitud_uuid es la clave primaria (SQLite informa la violación de una PK de texto como UNIQUE). Como solo se evalúa PENDIENTE, repetir la simulación no reevalúa nada.

Dónde iría la protección real (parada 8): en el servidor. El endpoint que reciba la solicitud debe guardar solicitud_uuid en una tabla con restricción UNIQUE **dentro de la misma transacción** que crea el pedido y descuenta el stock. Así, un reenvío después de un corte devuelve el resultado ya registrado en lugar de vender dos veces. La PK de la tablet solo evita duplicados locales.

```
control   |  antes  |  ahora  | sin_cambios
-------------+---------+---------+-------------
 pedidos     |  600000 |  600000 | t
 detalles    | 1500000 | 1500000 | t
 stock total | 3195000 | 3195000 | t
(3 rows)

                    control                    | filas
-----------------------------------------------+-------
 productos con stock distinto de la línea base |     0
(1 row)

 id_producto | precio_lista | stock | dado_de_baja
-------------+--------------+-------+--------------
         101 |      4028.85 |   469 | f
         106 |      3872.00 |   164 | f
         109 |       633.00 |   171 | t
         110 |       561.60 |   190 | f
         111 |      2057.00 |   259 | f
(5 rows)
```

Control final en PostgreSQL (servidor/99_control_final.sql): 600.000 pedidos y el mismo stock que la línea base, producto por producto. Solo cambiaron el precio del 101 (C2) y la baja del 109 (C4), preparados a mano como servidor. SIMULADO_APROBADO no creó ventas.

3.a Mapa de la capa de datos

Motor	Qué guarda	Quién manda sobre cada dato
PostgreSQL 16 (foodstore)	Catálogo, precios, stock, clientes, pedidos y detalles, usuarios y auditoría (TPI y Unidades 1 a 6).	**Manda sobre todo.** Precio, stock y ventas se deciden acá. Es el único sistema de registro.
MongoDB 8.0 (foodstore_nosql.pedidos, Unidad 7)	Copia documental de 12 pedidos con cliente y líneas embebidas, precios históricos en Decimal128.	**No manda.** Es una proyección de lectura que se regenera desde PostgreSQL (tp9/scripts/exportar_migracion.py). Si difiere, gana PostgreSQL.
SQLite (foodstore_movil.db, tablet)	producto_cache (copia fechada), snapshot_servidor, borradores, ítems y cola_sync.	**Manda solo sobre sus borradores y su cola.** El caché obedece a la última instantánea. SIMULADO_APROBADO es una propuesta, no una venta.

Flujo: PostgreSQL → (export) → MongoDB, solo lectura. PostgreSQL → (v_snapshot_movil, CSV) → SQLite. SQLite → (cola_sync, propuesta) → PostgreSQL decide. Ninguna flecha escribe precio, stock ni ventas desde un motor secundario.

3.b Plan de mantenimiento (agregado al de la Unidad 6)

Pregunta	Respuesta
¿Qué se respalda y cómo se prueba la restauración?	PostgreSQL: pg_dump -Fc diario con restauración cronometrada en foodstore_restore_test (tp8/backup_foodstore.sh y restore_test.sh; RPO 24 h). MongoDB: no se respalda, porque se regenera desde PostgreSQL. Tablet: al cierre de cada feria, `sqlite3 foodstore_movil.db ".backup respaldo_tablet.db"`, antes de sincronizar. Se prueba abriendo la copia con `PRAGMA integrity_check` (ok), `foreign_key_check` (sin filas) y comparando la cola por estado: 1 CONFLICTO, 2 RECHAZADO y 3 SIMULADO_APROBADO en ambas (evidencia/07_respaldo_tablet.txt).
Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?	No le envían nada al servidor como verdad. Descartan su instantánea, porque puede ser más nueva que el servidor restaurado, y bajan una nueva. Las solicitudes evaluadas contra una copia con copiado_en posterior a la fecha del respaldo se reenvían como solicitudes nuevas (UUID nuevo), para que el semáforo las juzgue contra el servidor restaurado. Los borradores no se pierden: viven en la tablet.
¿Cada cuánto y quién revisa CONFLICTO o RECHAZADO?	Al cierre de cada jornada de feria, y antes de la sincronización real, el responsable del food truck (rol responsable). CONFLICTO: se contacta al cliente; si acepta, solicitud nueva con el precio actual, como en C2. RECHAZADO: el borrador se anula o se rearma sin el producto. La consulta de revisión está en evidencia/07_respaldo_tablet.txt.

3.c Code review de los scripts de la cursada con IA

La consigna pide OpenCode en el agente Plan. No estaba disponible, así que la revisión se hizo con Claude (Claude Code) sobre los scripts de las Unidades 1 a 8 (ver docs/bitacora_ia.md). Cada hallazgo se comprobó en la base; la salida está en evidencia/06_code_review.txt.

#	Unidad / script	Hallazgo	Comprobación en la base	Decisión y motivo
H1	U7 - tp9/consultas_postgresql.sql §1	La consulta de claves une constraint_column_usage y multiplica filas: la PK compuesta sale como «id_pedido, id_pedido, id_producto, id_producto».	Versión de main contra versión corregida, sobre foodstore: 4 columnas antes y «id_pedido, id_producto» después.	**Corregido** en la rama tp-u7: la tabla referenciada se busca en una subconsulta. Las cifras del informe de la Unidad 7 no cambian.
H2	U1 - sql/01_restricciones_integridad.sql	Falla sobre una base creada con schema.sql: intenta crear restricciones que ya existen.	Dentro de BEGIN … ROLLBACK, sobre la copia: «constraint ck_producto_precio_positivo … already exists».	**Aceptado** y documentado, sin modificar el entregable del TP2: el script es para el esquema anterior al TPI, y schema.sql ya incorporó esas reglas.
H3	U8 - servidor/01_v_snapshot_movil.sql	Propuesta: usar round(precio_lista × 100) en lugar del cast, por si se pierden centavos.	precio_lista es NUMERIC(12,2): 0 de 10.000 productos tienen fracción de centavo y el cast coincide con round() en todos.	**Descartado:** con escala 2, × 100 es un entero exacto. round() no agrega nada.

#	Unidad / script	Hallazgo	Comprobación en la base	Decisión y motivo
H4	U8 - movil/cargar_snapshot.sql	Propuesta: recargar producto_cache con DELETE + INSERT, más simple que el UPSERT.	En una copia de la tablet: «FOREIGN KEY constraint failed». Hay 7 productos referenciados por borradores y 1 lápida.	**Descartado:** rompería la FK de los borradores y perdería las lápidas que C4 necesita.

Archivos: movil/schema_sqlite.sql, pruebas_sqlite.sql, cargar_snapshot.sql, semaforo.sql, casos/, servidor_snapshot.csv y foodstore_movil.db; servidor/; run_tp10.sh; docs/kiro/, bitacora_ia.md y DUIA.md; evidencia/ (salidas completas).